

ODD 2024
Tutorial Sheet – 6 & 7
Software Development Fundamentals – I (15B11CI111)

COURSE OUTCOMES		COGNITIVE LEVELS
CO1	Explain the logic for solving problems considering various phases of software development life cycle and depicting them using algorithms and flowcharts	Understand (Level 2)
CO2	Explain basics of C programming concepts to make decision for solving problems	Understand (Level 2)
CO3	Demonstrate and contrast different methods for writing modular programs in C	Understand (Level 2)
CO4	Use various C programming constructs to implement iteration, and recursion	Apply (Level 3)
CO5	Apply and implement arrays, pointers, structures and file handling for solving real-world problems	Apply (Level 3)

Note: Students are advised to submit their solutions to their respective tutorial faculties

Prog 1: Write a C program to count total number of positive and negative elements in an array.

Input/Output:

Input: arr[] = {2, -1, 5, 6, 0, -3}

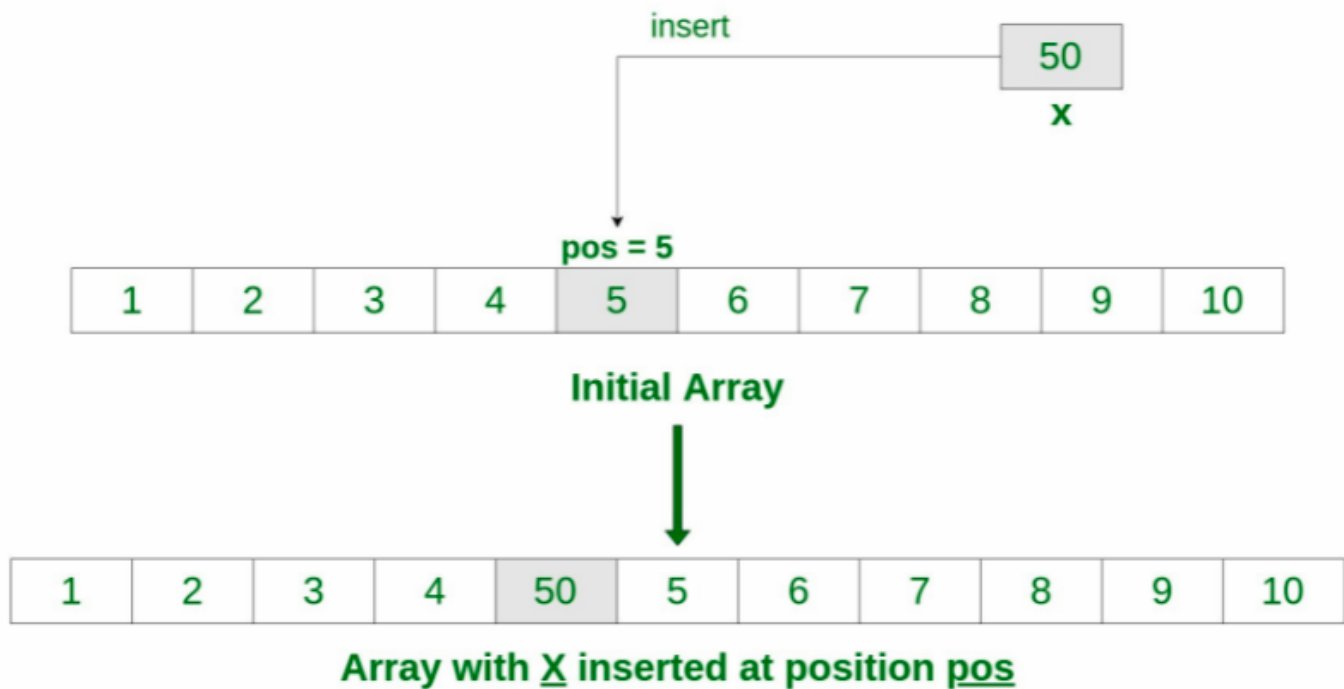
Output:

Positive elements = 3

Negative elements = 2

There are 3 positive, 2 negative, and 1 zero.

Insert an element at a specific position in an Array.



Prog 2: Write a C program to insert an element in an array. (starting, ending and mid)

Prog 3: Write a C program to delete an element from an array at specified position.

This C program is to delete an element from an array from a specified location/position. For example, if an array a consists of elements $a=\{7,8,12,3,9\}$ and if we want to delete element at position 3 then the new array would be $a=\{7,8,12,9\}$ (as array starts from index 0).

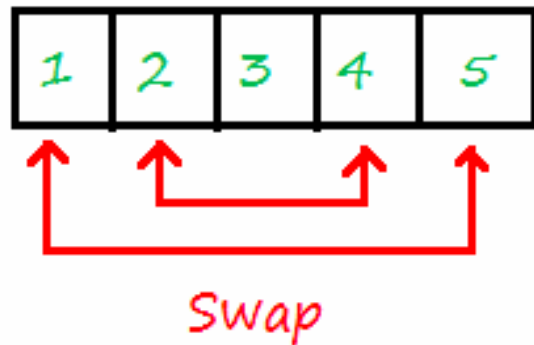
We start to iterate from the position from which we want to delete the element. The reason for this is so that all the elements after the element which is deleted will be shifted by one place towards the left.

Prog 4: Consider an array `arr[]` which consists of n numbers. WAP to:

- Reverse the array `arr[]`.
- Store the reverse of array `arr[]` In another array `arr2[]`.
- Print reverse of the array `arr[]` (without using another array).

The logic of reversing an array without using another array is to swap elements of the first half with elements of the second half i.e. the first element with the last element, then the second element with the second last element, and so on, till the middle element of the array. In programming, we use while loop with the condition $i < j$ to implement the process described above. Because if $i=j$, it means it is the middle element of the array and the middle element doesn't need to be swapped.

Original Array -



Reversed Array -



Prog 5: Consider an array marks[] which stores marks of n students. WAP to:

- a) Store and print marks of n students in array marks[].
- b) Find the highest and lowest marks
- c) Find the average, median, mode of marks.
- d) Create another array marks2 [] which stores the elements in marks[] array in non-decreasing order.

Steps:

- 1. Create an array marks2 of fixed size (maximum capacity), lets say 20.
- 2. Copy elements of marks[] array into marks2[] array.
- 3. Take i, a variable which stores the number of elements of the array, less than maximum capacity of array.
- 4. Iterate via for loop to take array elements as input, and print them.
- 5. The array elements are in unsorted fashion, to sort them, make a nested loop.
- 6. In the nested loop, the each element will be compared to all the elements below it.
- 7. In case the element is greater than the element present below it, then they are interchanged
- 8. After executing the nested loop, we will obtain an array in ascending order arranged elements.

- e) Create another array marks3 [] which stores the unique elements in marks [] array.

Steps:

Given an unsorted array of length N, we have to print the unique elements of array. If there are three occurrences of 5 then we have to print 5 only once. We can find the unique element in an array by traversing the array from index 0 to N-1 and for each element again traversing the array to find any duplicated element.